InfraPulse

Photo-Based Defect Detection & Priority Maintenance Web System

Team Shauryas · Takneek'26

3 September 2026

1 Approach

The problem is a triage problem wearing a computer-vision costume. A resident photographs a defect; the system must decide *what* it is, *who* fixes it, and *how soon* — with no human sorting the pile. We built it as four stages, each one feeding the next:

photograph → classify → route to category → score priority → place in queue

Stage 1 — Classification. A convolutional neural network fine-tuned on four defect classes. We chose image classification over object detection deliberately: the problem statement asks us to *identify the visible defect*, not to localise every instance of it. Classification needs weaker labels, trains faster, and is more robust on the small datasets publicly available for these defects.

Stage 2 — Routing. A fixed lookup from defect to maintenance category, given by the problem statement. No learning required, and nothing that can drift.

Stage 3 — Priority. Measured from the photograph itself — how bad the defect looks, and how much of the frame it occupies. Detailed in Section 3.

Stage 4 — Queue. Each category owns a queue ordered by priority score. New complaints find their position on insertion; nothing is re-sorted by hand.

Technology. Python throughout. PyTorch for the model, FastAPI with server-rendered Jinja2 templates for the web application, SQLite for storage, OpenCV and Pillow for image measurement. One deployable process serves both portals and the API, which keeps the hosted system simple enough to be reliably available during evaluation.

Compliance. Inference runs inside our own application, on CPU, using weights we trained. No external AI/ML inference service is contacted at any point. The only pretrained starting point is ResNet18 pretrained on ImageNet — a general-purpose dataset explicitly permitted as a backbone. No checkpoint pretrained on defect, crack, stagnant-water or building-damage data is used anywhere in the pipeline.

2 Detection Logic

2.1 Dataset Construction

No single public dataset covers all four required classes, so we assembled one from five sources and mapped their labels onto ours explicitly.

Source	Contributes	Notes
BD3 (Kaggle)	spalling, paint peeling	Smartphone photos, ∼1 m from walls, 50+ campus buildings
Roboflow `stagnant-water`	stagnant water	Real puddles, outdoor, CC BY 4.0
Roboflow `sep`	spalling, tile cracks	Building-surface defects, public domain
Roboflow `internal-wall-defect`	paint peeling	Interior walls
Roboflow `tile-classification`, `building-surface`	tile cracks	Building context

Two decisions here mattered more than anything else in the pipeline.

Explicit label mapping, not keyword matching. Our first attempt matched source labels by keyword. The rule `"tile"` → `cracked_tiles` silently swept in 1,125 boxes labelled `tile loss` — a *missing* tile is a hole in the wall, not a cracked one — making 31% of that class wrong. The same rule nearly pulled `water seepage` (damp staining on a wall) into `stagnant_water`. We replaced the heuristic with a label-by-label map and excluded both.

Cropping detection boxes. The Roboflow sources are object-detection datasets. We crop each annotated box with 25% padding rather than using whole images. This gives the network a clean close-up of the defect and strips background that would otherwise let it recognise *which dataset* a photo came from instead of *what defect it shows*.

Final dataset: **5,418 images** across the four classes.

2.2 Preventing Leakage — the Measurement That Changed Our Numbers

Because many images are crops of the same original photograph, a naive random split puts one crop of a photo in training and another in testing. The model has then effectively seen the test image, and the reported score is fiction.

We split by **source photograph**: every crop of a given photo lands in exactly one of train/val/test, together. The split code asserts that zero source photographs appear in more than one partition.

This turned out to be necessary in a way we did not initially detect. Roboflow exports augmented twins of each photo — rotated, flipped, recoloured — under *different filenames* (`<stem>_jpg.rf.<hash>.jpg`). Measured duplication:

Source	Files	Distinct originals	Ratio
`sep`	4,610	1,670	2.8×
`tileB`	459	153	3.0×
`wall`	4,442	2,798	1.6×
`water`	1,836	1,836	1.0×

Grouping on the whole filename would have treated those twins as different photographs. We group on the stem before `.rf.`, and additionally cap each source photograph at three crops so no single photo floods a class with near-identical images.

Split: **3,782 train / 817 validation / 819 test** (70/15/15 by source photograph).

2.3 Model and Training

- **Backbone:** ResNet18, ImageNet-pretrained. The final fully-connected layer is replaced with a fresh 4-output layer; all layers are then fine-tuned.
- **Differential learning rates:** backbone 3×10^{-5}, new head 1×10^{-3}, under a OneCycle schedule. The pretrained features get nudged, not overwritten.
- **Augmentation:** random resized crop, horizontal and vertical flips, $\pm 20^{\circ}$ rotation, colour jitter, random erasing. Validation and test see only resize and centre crop.
- **Class balance:** a weighted sampler shows rarer classes more often.
- **Loss:** cross-entropy with 0.05 label smoothing. **Optimiser:** AdamW.
- **Selection:** 14 epochs, keeping the epoch with the highest *validation macro-F1* rather than the last epoch.

Trained on a single T4 GPU in approximately 8 minutes. Exported checkpoint: 44.8 MB.

2.4 Serving

At request time the application resizes the photograph, normalises with ImageNet statistics, and runs a forward pass. The predicted class gives the defect name; a fixed lookup gives the category. Both are displayed on the user and staff portals without any manual selection.

3 Priority-Ranking Method

3.1 The Formula

$$\boxed{\text{priority\_score} = 100 \times \text{base\_weight} \times (0.5 \times \text{severity} + 0.5 \times \text{extent})}$$

Ties are broken by submission time, oldest first. `app/ml/priority.py` is the single source of truth, and the queue sorts by exactly the number it returns.

3.2 Visible Extent — via Grad-CAM on Our Own Network

We need to know how much of the photograph the defect occupies, without training a second model or obtaining segmentation labels.

Grad-CAM gives us this for free. We hook the last convolutional block, take the gradient of the predicted class score with respect to those feature maps, weight each channel by its mean gradient, and sum. The result is a heatmap of the pixels that drove the decision. Thresholding it at 0.5 and taking the highlighted fraction of the image gives **extent** $\in [0, 1]$.

The elegance is that it reuses the classifier we already trained — the same network that says *what* the defect is also tells us *where* it is.

3.3 Visible Severity — measured inside that region

Three signals, computed only within the Grad-CAM region:

Signal	Weight	Rationale
Local contrast	0.45	A deep spall with exposed material varies far more in brightness than a flat stain
Edge density	0.35	Fragmentation, multiple crack lines and flaking edges all raise this
Intensity deviation	0.20	How sharply the region stands out from the surrounding surface

Each is scaled to $[0, 1]$ with a fixed divisor, then combined. **severity** $\in [0, 1]$.

3.4 Base Weights

Defect	Weight	Reason
Spalling	1.00	Alone in Structural
Stagnant water	1.00	Alone in Functional
Cracked tiles	1.00	Performance — problem statement fixes cracked tiles above paint peeling
Paint peeling	0.75	Performance — the lower of the two

Because each category owns a separate queue, a base weight only ever affects ordering *within* a category. Performance is the only category holding two defect types, which is exactly where the problem statement specifies an ordering.

3.5 Properties

- **Deterministic.** The same photograph always produces the same score.
- **Computed live.** Every input comes from the submitted image at request time. Nothing is memorised, hardcoded, or pre-associated with any expected evaluation input.
- **Transparent.** Each complaint stores its own arithmetic as a plain-English string, shown in the interface, so a reader can see *why* a complaint sits where it does.

3.6 Worked Examples from the Running System

Complaint	Defect	Severity	Extent	Score	Band
Concrete broken away, rebar exposed	Spalling	0.796	0.220	**50.83**	High
Crack network across ceramic surface	Cracked Tiles	0.392	0.544	**46.85**	High
Floor tiles cracked, fragment missing	Cracked Tiles	0.406	0.380	**39.27**	Medium
Single paint flake lifting	Paint Peeling	0.109	0.291	**14.99**	Low

The exposed-rebar case scores 3.4× the loose paint flake. That ordering was produced by measurement, not by any rule naming those defects.

3.7 Deliberately Excluded: Age-Based Escalation

Letting old complaints climb the queue over time is realistic and prevents indefinite starvation. We left it out because it would make queue order disagree with visible severity, which is what the system is required to rank by. Submission time acts as a tie-breaker only.

See Section 6 for how we would add it properly.

4 Evaluation Results

Measured on the **819-image held-out test set**, split by source photograph so no photograph contributes to both training and testing.

Metric	Value
Test accuracy	**98.05%**
Macro F1	**97.86%**
Best validation macro F1	97.80%

4.1 Per Class

Class	Precision	Recall	F1	Support
Stagnant water	0.9919	1.0000	**0.9959**	245
Paint peeling	0.9732	0.9909	**0.9820**	220
Cracked tiles	0.9833	0.9779	**0.9806**	181
Spalling	0.9702	0.9422	**0.9560**	173

4.2 Confusion Matrix

Rows are the true class, columns the prediction.

	cracked_tiles	paint_peeling	spalling	stagnant_water
cracked_tiles	**177**	1	3	0
paint_peeling	0	**218**	2	0
spalling	3	5	**163**	2
stagnant_water	0	0	0	**245**

4.3 Reading the Errors

Spalling is the weakest class (F1 0.956), and its dominant error is **5 spalling images read as paint peeling**, with 3 the other way. This is the physically sensible confusion: both defects are a surface layer coming away from a wall, and separating them depends on *depth*, which a single photograph barely carries.

We regard this error pattern as evidence the model is genuinely looking at the defect. A model exploiting dataset artefacts would fail in scattered, meaningless ways rather than concentrating its mistakes on the one pair a human also finds ambiguous.

Stagnant water is perfectly separated (245/245). Water is glossy and reflective and shares no visual vocabulary with dry masonry.

4.4 On the Honesty of These Numbers

Our first run reported 98.03% on a *randomly* split test set. We did not trust it, and built the grouped split and duplicate detection described in Section 2.2 specifically to test whether it was inflated. After grouping augmented twins and capping crops per photograph, the score held at 98.05%. The number survived an attempt to break it, which is the only reason we quote it.

5 Limitations

Source concentration. Each class draws predominantly from one or two datasets. Part of the measured accuracy may therefore reflect the model recognising *which dataset* an image resembles rather than which defect it shows. Cropping to defect regions and heavy augmentation mitigate this but cannot eliminate it. It is the single largest caveat on the numbers in Section 4.

Tile domain mismatch. Our cracked-tile images are predominantly small mosaic *facade* tiles. Testing on a photograph of damaged tile skirting in a hostel corridor, the model returned Paint Peeling at 46.3% confidence — low confidence, but the wrong label. Large interior floor tiles are under-represented in the training distribution.

Closed-set classification. The system must assign one of four labels. Photographs of defects outside that set are forced into the nearest class. Two water-stain images in our testing were classified as Paint Peeling at 95–97% confidence, because "damp stain" is not an available answer.

Severity is a visual proxy. Contrast, edge density and intensity deviation are correlates of severity, not measurements of it. A single photograph cannot establish structural depth, and a shallow but high-contrast defect can outscore a deep but uniform one.

Grad-CAM localisation is coarse. The heatmap derives from a 7×7 feature map upsampled to full resolution, so extent is approximate for small or thin defects such as hairline cracks.

No confidence gate. Predictions are surfaced identically whether the model is 98% or 46% confident. Low-confidence cases are routed with the same authority as certain ones.

Ephemeral storage. Uploaded photographs and the SQLite database live on the host's local disk and do not survive a container restart on a free hosting tier.

6 Suggestions for Improving Accuracy

- **Collect in-domain photographs.** The highest-value improvement available. A few hundred photographs of defects in the actual buildings the system will serve — hostels, academic blocks, corridors — would address both the source-concentration and tile-mismatch limitations directly. Public datasets were assembled for other buildings in other countries; the deployment environment is the distribution that matters.
- **Surface a confidence threshold.** Below roughly 60%, display the top two candidates and flag the complaint for staff confirmation rather than asserting a single answer. This converts a silent error into a visible question, and would have caught the 46.3% tile case.
- **Add an out-of-scope class.** Training a fifth "other defect" class on stains, algae, cracks and normal walls — all available in BD3 — would stop non-target defects being forced

into one of the four categories.

- **Test-time augmentation.** Averaging predictions over an image and its horizontal flip typically recovers one to two points of accuracy for a few milliseconds of extra compute.
- **Benchmark larger backbones.** ResNet50 and EfficientNet-B0 under an identical protocol would establish whether ResNet18 is capacity-limited. Our schedule did not permit the comparison.
- **Learn severity rather than infer it.** With a few hundred complaints rated by maintenance staff, severity could be regressed directly from the image instead of estimated from contrast and edge statistics — replacing a documented heuristic with a fitted one.
- **Age-based escalation.** Add a bounded term that lets a complaint climb slowly with age, so low-priority items cannot starve indefinitely, while keeping visible severity dominant in the ordering.

7 Challenges and How We Solved Them

Challenge	Resolution
No dataset covers all four classes	Assembled from five public sources with an explicit label map
Keyword label matching pulled in `tile loss` and `water seepage`	Replaced with label-by-label mapping; excluded both
Crops of one photo split across train and test	Grouped split by source photograph, with an assertion that it holds
Roboflow augmented twins under different filenames	Group on the filename stem before `.rf.`; cap 3 crops per photograph
Measuring defect extent without segmentation labels	Grad-CAM on our own classifier, thresholded for area
PyTorch's CUDA wheel too large for free hosting	Install CPU-only wheels explicitly before the requirements file
Scheduler exhaustion on re-running training	Rebuild optimiser and scheduler with the model each run

8 Citations

Datasets

- Kottari, P. and Arjunan, P. *BD3: Building Defects Detection Dataset.* BuildSys '24. DOI: 10.1145/3671127.3698789
- Roboflow Universe: `stagnant-water/stagnant-water` (CC BY 4.0); `joe-i4soa/sep` (public domain); `chew-poh-yee/internal-wall-defect`; `cognate-prqm5/tile-classification-34erw` `hku-sdauc/building-surface`

Models

- ResNet18 pretrained on ImageNet, via `torchvision.models` — general-purpose backbone only

Libraries

PyTorch, torchvision, FastAPI, Uvicorn, Starlette, Jinja2, SQLite, OpenCV, Pillow, NumPy,

scikit-learn, Matplotlib. All used for training, serving, storage and image measurement — none performs defect detection or classification on our behalf.